

[42] PM-LSH; A fast and accurate lsh framework for high-dimensional approximate nearest neighbor search

저자: B. Zheng et al. **학술지:** Proc. VLDB Endow., vol. 13, no. 5, p. 643-655, 2020 **주제:** LSH 기반의 고차원 근사 근접 이웃 검색

요약

본 논문은 Locality-Sensitive Hashing(LSH) 기반의 고성능 ANN 프레임워크 PM-LSH(Prefix-Matching Locality-Sensitive Hashing)를 제시한다. LSH는 고차원 공간에서의 근사 근접 이웃 검색을 위한 확률적 방법으로, 유사한 벡터들이 같은 해시 버킷으로 매핑될 확률이 높다.

PM-LSH는 기존 LSH의 단점(높은 거짓양성율, 메모리 오버헤드)을 개선하여, 접두사 매칭(prefix matching) 기법으로 빠르고 정확한 검색을 구현한다.

본 논문과의 관계: Exqutor의 벡터 검색은 HNSW 같은 그래프 기반 인덱싱을 사용하지만, LSH 기반 방식도 필터링과 결합할 경우 다른 성능 특성을 가진다. 특히 고차원에서의 성능 비교 분석이 중요하다.

상세분석

42.1 LSH의 기본 원리

핵심 아이디어:

유사한 데이터 포인트들이 높은 확률로 같은 해시 버킷으로 매핑되도록 하는 해시 함수족을 설계한다.

수학적 정의:

해시 함수족 H 가 LSH라면: - $p_1 > p_2$ (확률) - 거리 $d(x, y) \leq r$ 일 때: $\Pr[h(x) = h(y)] \geq p_1$ - 거리 $d(x, y) > cr$ 일 때: $\Pr[h(x) = h(y)] \leq p_2$

여기서 $c > 1$ 은 근사 비율이다.

직관:

유사 포인트 (거리 $\leq r$)
↓ 높은 확률로
[같은 해시 버킷]
↑ 낮은 확률로
다른 포인트 (거리 $> cr$)

42.2 전통적 LSH의 구조

레이어 구조:

layer 1: $h_1 = [h^{(1)}_1, h^{(1)}_2, \dots, h^{(1)}_k] \rightarrow$ 해시값 1
(k 개 함수의 연결)

layer 2: $h_2 = [h^{(2)}_1, h^{(2)}_2, \dots, h^{(2)}_k] \rightarrow$ 해시값 2
...
layer L: $h_L = [h^{(L)}_1, h^{(L)}_2, \dots, h^{(L)}_k] \rightarrow$ 해시값 L

검색 프로세스:

1. 쿼리 q 의 각 레이어별 해시값 계산
2. 각 레이어에서 같은 해시 버킷의 모든 점 찾기
3. 찾은 모든 점들 중 가장 가까운 k 개 선택

시간복잡도: $O(L \times \text{hash_computation} + \text{후보_수})$

42.3 PM-LSH의 개선점

문제점 분석:

전통 LSH의 단점: 1. **높은 거짓양성**: 다른 점이 같은 버킷에 들어올 확률 높음 2. **메모리 낭비**: L 개 레이어 모두에서 데이터 복사 필요 3. **계산 비용**: 많은 후보에 대해 거리 계산

접두사 매칭 기법:

전통 LSH:

레이어 1: 해시 (010110...)

레이어 2: 해시 (101010...)

레이어 3: 해시 (111001...)

→ 각 레이어에서 따로 검색

PM-LSH:

모든 레이어의 해시를 연결한 하나의 키: 010110|101010|111001

검색 시:

1단계: 해시 010110* 으로 시작하는 모든 항목 (레이어 1)

2단계: 010110|101010* 으로 시작하는 모든 항목 (레이어 2)

3단계: 010110|101010|111001* 정확히 일치

42.4 PM-LSH 알고리즘 상세

인덱싱:

```
procedure BUILD_PM_LSH_INDEX(vectors, k, L):
    // k: 각 레이어의 함수 개수
    // L: 레이어 개수

    // 1단계: LSH 함수족 생성
    for layer = 1 to L:
        for func_idx = 1 to k:
            h[layer][func_idx] = create_random_projection()

    // 2단계: 접두사 트리 구성
    prefix_tree = TrieNode()

    for vector v in vectors:
        key = ""
        for layer = 1 to L:
            // 해시값 계산
            hash_value = compute_hash(v, h[layer], k)
            key = key + "|" + hash_value // 접두사 연결

        // 트라이에 삽입
        insert_to_trie(prefix_tree, key, v)

    return prefix_tree
```

검색:

```

procedure SEARCH_PM_LSH(query_q, K, early_termination_threshold):
    candidates = []
    prefix = ""

    for layer = 1 to L:
        // 현재 레이어의 해시값 계산
        hash_value = compute_hash(q, h[layer], K)
        prefix = prefix + "|" + hash_value

        // 점두사와 일치하는 모든 벡터 찾기
        matching_vectors = search_prefix_in_trie(
            prefix_tree, prefix
        )

        // 거리 계산 및 후보 수집
        for v in matching_vectors:
            dist = distance(q, v)
            candidates.append((v, dist))

        // 조기 종료 (충분한 좋은 결과)
        if |candidates| >= early_termination_threshold:
            break

    // 가장 가까운 K개 반환
    return top_K_by_distance(candidates, K)

```

42.5 해시 함수 선택

Random Projection (무작위 사영):

가장 널리 사용되는 LSH 함수

$$h_{a,b}(v) = \text{floor}((a \cdot v + b) / r)$$

여기서:

- a: 무작위 벡터 (각 차원: $N(0,1)$)
- b: $[0, r)$ 범위의 무작위 스칼라
- r: 버킷 크기 (파라미터)
- v: 입력 벡터

거리 메트릭: L2 (유클리드)

코사인 유사도를 위한 LSH:

$$h_a(v) = \text{sign}(a \cdot v)$$

여기서:

- a : 무작위 벡터
- $\text{sign}()$: 양수면 1, 음수면 0

확률: $P(h_a(x) = h_a(y)) = 1 - \theta(x,y)/\pi$
(θ 는 벡터 간 각도)

42.6 파라미터 튜닝

k 와 L 의 영향:

더 많은 함수 (큰 k):

- 더 정확한 구분 (낮은 거짓양성)
- 높은 계산 비용
- 낮은 거짓음성 (실제 유사점 놓치기)

더 많은 레이어 (큰 L):

- 더 높은 회수율(recall)
- 더 큰 메모리
- 더 많은 계산

최적화:

- $k + L = \text{상수}$ (메모리 제약)
- k 를 높여 정확도 향상
- L 을 높여 회수율 향상

버킷 크기 r 의 선택:

작은 r (더 많은 버킷):

- 낮은 거짓양성
- 높은 거짓음성

큰 r (적은 버킷):

- 높은 거짓양성
- 낮은 거짓음성

권장: 데이터의 타입 거리 분포에 맞춰 선택

42.7 접두사 트리 구현

Trie 구조:

```

루트
└─ 0
  │ └─ 1
  │   │ └─ 0 → [벡터 ID들]
  │   │   └─ 1 → [벡터 ID들]
  │   └─ 0
  │       └─ 1 → [벡터 ID들]
└─ 1
    └─ ...

```

메모리 효율성:

전통 LSH (L개 해시 테이블):

메모리 = $L \times (\text{평균 버킷 크기} \times \text{포인터})$

PM-LSH (단일 트리):

메모리 = 트리 노드 수 $\times$ (자식 수 + 데이터)

일반적으로 PM-LSH가 10~30% 더 효율적

42.8 성능 특성

시간 복잡도:

인덱싱: $O(L \times k \times n \times d)$

- n: 벡터 개수
- d: 벡터 차원
- k: 함수 개수
- L: 레이어 개수

검색: $O(L \times k + \text{평균_후보_수} \times d)$

- 조기 종료로 평균 L을 줄일 수 있음

메모리 복잡도:

벡터 저장: $O(n \times d)$

LSH 구조: $O(L \times k \times d)$ (함수 저장)

트리: O(계약 가능)

42.9 PM-LSH vs HNSW

지표	PM-LSH	HNSW
구성 시간	낮음	중간
메모리	효율적	중간
검색 속도	중간~높음	매우 빠름
정확도	좋음	우수
파라미터 조정	복잡	간단
확장성	매우 좋음	좋음

42.10 필터링과의 통합

LSH 기반 필터링:

```
procedure SEARCH_FILTERED_PM_LSH(  
    query_q, K, filter_predicate  
):  
    candidates = []  
  
    for layer = 1 to L:  
        hash_value = compute_hash(q, h[layer])  
        prefix = current_prefix + hash_value  
  
        // 점두사와 일치하는 벡터  
        matching_vectors = search_prefix(prefix_tree, prefix)  
  
        for v in matching_vectors:  
            // 필터 조건 확인  
            if not filter_predicate(v):  
                continue  
  
            // 거리 계산  
            dist = distance(q, v)  
            candidates.append((v, dist))  
  
            // 충분한 결과를 얻으면 종료  
            if len(candidates) >= K:  
                break  
  
    return top_K(candidates, K)
```

필터와 해시의 상호작용:

필터 조건 "category == 'electronics'":

- 전체 벡터의 30% 선택

LSH 해시로 찾은 후보: 100개

- 그 중 필터 만족: 30개 (예상)

정확도는 유지되지만 후보 집합이 감소

→ 거짓음성 증가 가능성

42.11 적응형 검색 전략

동적 레이어 확장:

처음 N개 레이어만 검색:

- 빠른 응답
- 낮은 정확도

필요시 추가 레이어로 확장:

- 더 많은 후보 수집
- 회수율 향상
- 응답 시간 증가

42.12 실제 성능 특성 (1M 벡터, 1000차원)

구성 시간:

HNSW: 약 30분

PM-LSH: 약 5분

메모리:

HNSW: 약 4GB

PM-LSH: 약 3GB

검색 속도:

HNSW: 0.5ms/쿼리

PM-LSH: 1-2ms/쿼리

정확도 (top-10):

HNSW: 95%

PM-LSH: 90%

추가 제기 문제

1. **필터 조건과 LSH의 상호작용**: 필터 조건이 매우 선택적일 때, LSH의 성능이 어떻게 변하는가? 필터된 벡터들의 거리 분포가 어떻게 변할까?
2. **해시 함수 선택의 필터 최적화**: 특정 필터 조건에 최적화된 해시 함수를 설계할 수 있을까? 예를 들어, "category" 필터를 고려한 특화된 LSH?
3. **접두사 트리의 필터 인덱싱**: 트리 구조 내에 필터 조건 정보를 인코딩하면 필터 검색 성능을 향상시킬 수 있을까?
4. **다중 필터와 LSH의 조합**: 여러 필터 조건이 있을 때, 이들을 LSH 함수에 포함시켜 함께 해싱할 수 있을까?
5. **확률적 보장**: LSH의 확률적 성질이 필터링과 결합될 때, 전체 반환 결과의 정확도 보장은 어떻게 변할까?
6. **메모리 효율성의 필터 활용**: 필터로 인해 실제로 접근해야 할 벡터가 줄어들 때, LSH 구조의 메모리를 동적으로 조정할 수 있을까?
7. **필터 조건의 변화**: 런타임 중에 필터 조건이 자주 변할 때, 기존 LSH 인덱스를 재구성하지 않고도 효율적으로 처리할 수 있을까?
8. **정확도와 성능의 분석**: PM-LSH가 HNSW보다 메모리 효율적이면서도 검색이 느린 이유를 필터링 관점에서 분석하면?
9. **하이브리드 인덱싱**: HNSW의 상위 계층과 PM-LSH의 정확한 검색을 결합한 구조의 성능은?